

Verteilte Datenhaltung

Worum geht es?

Die Folien behandeln, wie große Datenmengen in **verteilten Systemen** gespeichert, partitioniert, repliziert und abgefragt werden. Schwerpunkt sind:

- **Big Data**
 - **verteilte Dateisysteme**
 - **NoSQL-Datenbanken**
 - **Replikation und Partitionierung**
 - **Vektordatenbanken für KI-Anwendungen**
-

1. Motivation

Klassische einzelne Datenbanken oder Dateisysteme reichen oft nicht mehr aus, wenn Daten:

- **sehr groß** sind
- **schnell wachsen**
- **unterschiedlich strukturiert** sind
- von **sehr vielen Nutzern** gleichzeitig verwendet werden
- in **Cloud-Umgebungen** flexibel skaliert werden sollen

Diese Anforderungen werden oft unter **Big Data** zusammengefasst.

Die typischen Eigenschaften von Big Data

Die Folien nennen die bekannten **V's**:

- **Volume**: große Datenmengen
 - **Velocity**: hohe Änderungs- und Verarbeitungsgeschwindigkeit
 - **Variety**: unterschiedliche Datenformen
 - **Veracity**: Unsicherheit / Qualität der Daten
-

2. Welche Technologien werden dafür genutzt?

Typische Technologien für verteilte Datenhaltung sind:

- **Cluster-Computing**

- **verteilte Dateisysteme** wie HDFS
 - **NoSQL-Datenbanken**
 - **verteilte Analyse- und Rechenframeworks** wie MapReduce, Spark oder Storm
-

3. Warum relationale Datenbanken allein oft nicht reichen

Relationale Datenbanken sind weiterhin nützlich, aber für viele Big-Data-Szenarien gibt es Probleme:

- horizontale Skalierung ist schwieriger
- starres Schema ist unflexibel
- sehr große Last und stark verteilte Daten sind unpraktisch
- für viele moderne Workloads braucht man andere Verteilungs- und Replikationsmodelle

Trotzdem sagt die Folie klar:

- relationale Datenbanken **bleiben wichtig**
 - oft nutzt man künftig **polyglotte Persistenz**, also mehrere Datenbankarten parallel
-

4. Was bedeutet NoSQL?

NoSQL ist kein ganz scharfer Begriff. Gemeint sind meistens Datenbanken, die für moderne verteilte Anforderungen gebaut wurden.

Typische Eigenschaften von NoSQL-Datenbanken:

- auf **Massendaten** ausgelegt
- **horizontal skalierbar**
- **verteilte Architektur**
- oft **schemafrei**
- einfache **Replikation**
- häufig eher **eventual consistent** als vollständig ACID-streng
- meist nicht primär relational organisiert

Wichtig:

NoSQL heißt sinngemäß eher „**Not Only SQL**“ als „kein SQL“.

5. Grundprinzipien: Partitionierung und Replikation

Das Kernprinzip verteilter Datenhaltung ist:

- **große Datenmengen aufteilen**
- **Teilbereiche mehrfach kopieren**

Dafür gibt es zwei zentrale Techniken:

Partitionierung

Daten werden in **Teilbereiche** zerlegt, damit sie auf mehrere Knoten verteilt gespeichert werden können.

Replikation

Diese Teilbereiche werden auf mehreren Knoten **kopiert**, damit:

- Ausfälle abgefangen werden
 - Last verteilt werden kann
 - Lesen und Schreiben skaliert werden können
-

6. Formen der Partitionierung

Funktionale Partitionierung

Daten werden nach fachlichen Gruppen getrennt, z. B.:

- Warenkatalog
- Lagerbestand
- Kundendaten

Horizontale Partitionierung (Sharding)

Der Datenbestand einer Art wird in **Untergruppen** zerlegt.
Ein Datensatz gehört immer genau zu **einem Shard**.

Zwei typische Varianten

Hash-basiertes Sharding

- Verteilung per Hashwert über den Schlüssel
- Vorteil: meist gleichmäßige Verteilung
- Nachteil: berücksichtigt Datenlokalität schlecht

Range-basiertes Sharding

- Verteilung nach Schlüsselbereichen

- Vorteil: Daten mit ähnlichen Werten können zusammenliegen
- Nachteil: Gefahr ungleicher Lastverteilung, daher evtl. Rebalancing nötig

Vertikale Partitionierung

Ein Datensatz wird entlang seiner **Attribute** aufgeteilt.

Beispiel: Produktbeschreibung und Lagerbestand werden getrennt gespeichert.

Das ist typisch für **spaltenbasierte Datenbanken**.

7. Rebalancing und Consistent Hashing

Wenn Knoten hinzukommen oder wegfallen, müssen Partitionen oft **umverteilt** werden.

Ziel:

- möglichst **wenig Daten verschieben**
- trotzdem die Last wieder gleichmäßig verteilen

Consistent Hashing

Hier werden sowohl:

- Serverknoten
- als auch Daten-Schlüssel

auf denselben **Hash-Ring** gelegt.

Ein Datensatz wird dann dem nächsten Knoten im Uhrzeigersinn zugeordnet.

Vorteil: Beim Hinzufügen oder Entfernen von Knoten müssen nur vergleichsweise wenige Daten umziehen.

8. Request Routing

Ein Client muss wissen, **auf welchem Knoten** seine Daten liegen.

Dafür braucht man Metadaten darüber:

- welche Partition wo liegt
- wie der Zugriff aktuell geroutet wird

Laut Folie gibt es drei Möglichkeiten:

- Routing in der Knotenschicht
- separater Routing-Layer

- Routing direkt durch den Client

Zur Verwaltung solcher Metadaten werden oft Koordinierungsdienste eingesetzt, etwa:

- **ZooKeeper**
 - **etcd**
 - **Chubby**
-

9. Replikation: Grundideen

Replikation bedeutet:

- dieselbe Partition liegt auf mehreren unabhängigen Knoten vor

Vorteile

- bessere Verfügbarkeit
- höhere Fehlertoleranz
- geringere Latenz
- Lastverteilung
- Skalierung von Read- und teils auch Write-Operationen

Die schwierige Frage ist:

Wie hält man mehrere Kopien konsistent, wenn sich Daten laufend ändern?

10. Leader-basierte Replikation

Das Standardmodell ist **Leader-Follower**:

- Schreiboperationen gehen an den **Leader**
- der Leader protokolliert die Änderung
- die Follower übernehmen diese Updates
- Lesezugriffe können oft auch auf Followern stattfinden

Vorteile

- klare Ordnung der Writes
- einfacher zu kontrollieren

Nachteile

- Leader ist ein möglicher **Bottleneck**

- bei Leader-Ausfall braucht man **Failover** und Leader-Wahl
-

11. Synchrone vs. asynchrone Replikation

Synchrone Replikation

Der Leader bestätigt einem Client ein Write erst dann, wenn alle Follower den Write bestätigt haben.

Vorteil

- nach erfolgreichem Write sind alle Replika aktuell

Nachteil

- geringe Verfügbarkeit
- Schreiboperation kann hängen, wenn Follower fehlen

Asynchrone Replikation

Der Leader bestätigt dem Client sofort und repliziert danach weiter.

Vorteil

- schneller
- verfügbarer

Nachteil

- Replika können zeitweise auseinanderlaufen
 - Trade-off zwischen Konsistenz und Verfügbarkeit
-

12. Umgang mit Ausfällen

Follower-Ausfall

Ein Follower kann nach Neustart über seinen persistenten Log wieder aufholen (**catch-up recovery**).

Leader-Ausfall

Dann braucht man:

- Ausfallerkennung

- Wahl eines neuen Leaders
- Rekonfiguration des Systems

Die Folien verweisen dabei auf Konsensverfahren wie **Raft** oder **Paxos**.

13. Multi-Leader-Replikation

Hier gibt es mehrere Leader gleichzeitig.

Das ist nützlich z. B. für:

- mehrere Rechenzentren
- bessere lokale Performanz
- Collaborative Editing
- Szenarien mit stärker verteilter Schreiblast

Problem

Wenn zwei Leader gleichzeitig widersprüchliche Updates annehmen, entstehen **Konflikte**.

Konfliktlösung

Die Folien nennen u. a.:

- **Last Writer Wins (LWW)**
 - **Anti-Entropy**
 - **MVCC** (mehrere Versionen, Anwendung entscheidet)
-

14. Leaderless Replication

Hier gibt es **keinen Leader**.

Clients lesen und schreiben direkt mit mehreren Replika.

Das ist das bekannte **Dynamo-Style-Prinzip**.

Eigenschaften

- kein Leader-Bottleneck
- kein klassisches Failover
- Konsistenzprobleme werden oft vom Client oder durch das System nachträglich behandelt
- Versionen oder logische Zeitstempel helfen, neuere Werte zu erkennen

15. CAP und PACELC

CAP-Theorem

Ein verteiltes Datensystem kann im Partitionierungsfall nicht gleichzeitig vollständig haben:

- **Consistency**
- **Availability**
- **Partition Tolerance**

Praktisch heißt das:

Bei Netzpartitionen muss man zwischen stärkerer Konsistenz und höherer Verfügbarkeit abwägen.

PACELC

PACELC erweitert CAP um die Frage:

Was passiert **ohne** Partition?

Dann lautet die Abwägung oft:

- **Consistency**
- oder **Latency**

16. ACID vs. BASE

ACID

Das klassische Modell relationaler Systeme:

- Atomicity
- Consistency
- Isolation
- Durability

BASE

Typisch für viele NoSQL-Systeme:

- Basically Available
- Soft State
- Eventually Consistent

Die Folie beschreibt BASE sinngemäß als pragmatischer, aber schwächer als ACID.

17. NoSQL-Datenmodelle und Aggregate

Die Folien betonen das Konzept der **Aggregate**.

Ein Aggregat ist:

- eine zusammengehörige Dateneinheit
- kann verschachtelte Strukturen, Arrays und Teilobjekte enthalten
- wird als **eine atomare Einheit** behandelt

Warum Aggregate wichtig sind

- sie definieren Grenzen für atomare Operationen
 - sie vereinfachen Konsistenz
 - sie helfen bei Verteilung im Cluster
 - ein Client bekommt die benötigten Daten möglichst von **einem Knoten**
-

18. Die vier Grundtypen von NoSQL-Datenbanken

1. Key-Value-Datenbanken

Sehr einfaches Modell:

- ein **Key**
- ein zugehöriger **Value**

Basisoperationen:

- `get`
- `put`
- `delete`

Vorteile

- sehr einfach
- sehr schnell
- gut für Sessions, Nutzerprofile, Warenkörbe, Caches

Nachteile

- ungeeignet für komplexe Abfragen
- schlecht für Beziehungen zwischen Daten
- keine starken Mehrfach-Transaktionen über viele Keys hinweg

Beispiel

- **Riak KV** als verteilte, leaderlose Key-Value-Datenbank mit eventueller Konsistenz
-

2. Dokument-orientierte Datenbanken

Das zentrale Objekt ist ein **Dokument**, z. B. als:

- JSON
- BSON
- XML

Ein Dokument:

- ist selbstbeschreibend
- kann verschachtelte Strukturen enthalten
- wird in Collections gespeichert
- braucht nicht überall dasselbe Schema

Beispiel

- **MongoDB**

MongoDB laut Folie

- Sharding + Replikation
 - standardmäßig eher **CP-orientiert**
 - Reads/Writes über Primary für starke Konsistenz
 - mit Reads von Secondaries nur noch eventual consistency
 - Replikation per **Raft**
 - unterstützt Multi-Document-ACID-Transaktionen mit Snapshot Isolation
-

3. Spaltenorientierte Datenbanken

Hier bestehen Datensätze aus:

- Zeilen mit Row-Key
- sehr vielen Spalten
- gruppiert in Spaltenfamilien

Das eignet sich gut für:

- Analyse-Workloads
- große, dünn besetzte Tabellen
- flexible Attributmengen

Beispiel

- **Google Bigtable**

Bigtable laut Folie

- verteilte, persistente, sortierte Map
 - Indexierung über (row_key, column_key, timestamp)
 - Strong Consistency in einem einzelnen Cluster
 - Eventual Consistency über mehrere replizierte Cluster
 - atomare Single-Row-Operationen
 - Koordination über **Chubby**
-

4. Graphdatenbanken

Graphdatenbanken modellieren:

- **Knoten** für Entitäten
- **Kanten** für Beziehungen
- beide können Properties haben

Sie eignen sich besonders für:

- komplexe Beziehungsstrukturen
- Traversierungen
- tiefe Beziehungshierarchien

Beispiel

- **Neo4j**

Neo4j laut Folie

- Causal Clustering
- Leader-Follower-Prinzip
- Writes über Leader, Reads über alle Knoten
- Replikation mit **Raft**
- Sharding über **Fabric**
- im Cluster eher kausale / eventuelle Konsistenz, standalone starke Konsistenz

19. Verteilte Dateisysteme

Verteilte Dateisysteme speichern sehr viele oder sehr große Dateien, die nicht mehr auf einen einzelnen Rechner passen.

Eigenschaften

- geeignet für Massendaten
 - Fehlertoleranz durch Replikation
 - dynamisch skalierbar
 - oft Grundlage für Data Lakes
 - teils mit Block-, Objekt- oder Dateisystem-Schnittstellen nutzbar
-

20. HDFS

Hadoop Distributed File System (HDFS) ist ein klassisches verteiltes Dateisystem.

Grundprinzip

- Dateien werden in große Blöcke geteilt
- Blöcke werden repliziert
- **Data Nodes** speichern die Daten
- **Name Node** verwaltet die Metadaten darüber, wo welche Blöcke liegen

Eigenschaften

- gut für sehr große Dateien
 - Kohärenzmodell: **write-once-read-many**
 - Data-Node-Ausfälle sind durch Replikation unkritisch
 - NameNode ist ein potenzieller Single Point of Failure, nutzt aber Recovery über Transaktionslog
-

21. Ceph

Ceph ist ein moderner verteilter Speicher, der verschiedene Schnittstellen bietet.

Laut Folien:

- **RADOS** als verteilter Objektspeicher
- **RBD** als Block Device

- **CephFS** als Dateisystem
- **Gateway** für S3/Swift-ähnliche APIs

Wichtige Eigenschaften

- Metadaten und Datenverwaltung sind getrennt
- alles ist replizierbar
- OSD-Daemons verwalten die eigentlichen Datenobjekte
- CRUSH bestimmt, wo Datenobjekte gespeichert werden
- Paxos wird für Koordination der Maps genutzt

Vorteil

Ceph ist flexibler als klassische Dateisysteme, weil es Objekt-, Block- und Dateisystemzugriffe kombiniert.

22. Vektordatenbanken und KI

Am Ende behandeln die Folien moderne **Vektordatenbanken**.

Was ist ein Embedding?

Ein Embedding ist ein **Vektor aus Zahlen**, der von einem KI-Modell erzeugt wird, um:

- Bedeutung
- Struktur
- Kontext
- Ähnlichkeit

eines Eingabeobjekts darzustellen.

Was macht eine Vektordatenbank?

Sie speichert, indiziert und durchsucht hochdimensionale Vektoren, um ähnliche Objekte zu finden.

Typische Einsatzfälle

- semantische Suche
- Empfehlungen
- Retrieval für LLMs
- Langzeitgedächtnis für Chatbots
- Cache für bereits beantwortete Fragen

23. Metriken für Ähnlichkeit

Vektordatenbanken nutzen verschiedene Distanz- bzw. Ähnlichkeitsmaße:

Cosine Similarity

- betrachtet den Winkel zwischen Vektoren
- gut für Textähnlichkeit

Dot Product

- berücksichtigt Winkel und Länge
- nützlich für Recommender-Szenarien

Euklidische Distanz

- absoluter Abstand im Raum
- oft teurer zu berechnen, daher Indexte und ANN nötig

24. ANN und Indexstrukturen

Da exakte Nachbarsuche bei sehr vielen hochdimensionalen Vektoren teuer ist, verwendet man **Approximate Nearest Neighbor (ANN)**.

Die Folien nennen typische Indexverfahren:

- **IVF** (Inverted File Index)
- **HNSW**
- **LSH**
- **PQ** (Product Quantization)

Kurz erklärt

IVF

Vektoren werden in Gruppen geclustert; gesucht wird nur in relevanten Gruppen.

HNSW

Mehrschichtiger Graph zur schnellen ANN-Suche.

LSH

Ähnliche Vektoren landen mit hoher Wahrscheinlichkeit in denselben Buckets.

PQ

Komprimiert Vektoren, spart Speicher und beschleunigt Suche.

25. Beispiele für Vektordatenbanken

Die Folien nennen u. a.:

- **Milvus**
- **Weaviate**
- **Pinecone**
- **Qdrant**
- graphbasierte Varianten mit HNSW
- hybride Systeme wie **Redis**, **PostgreSQL mit pgvector**, **MongoDB Atlas Search**

Außerdem werden Cloud-Ökosysteme erwähnt, etwa:

- Amazon Kendra
 - Google Vertex AI Matching Engine
 - Azure Cognitive Search
-

26. Kurzfazit

Zentrale Idee

Verteilte Datenhaltung löst das Problem, dass Datenmengen und Lasten für einzelne Systeme zu groß werden.

Die wichtigsten Werkzeuge

- **Partitionierung**
- **Replikation**
- **Sharding**
- **Routing**
- **Rebalancing**
- **Konsistenz-Trade-offs**

NoSQL ist kein Ersatz für alles

NoSQL-Datenbanken sind für viele verteilte und skalierende Szenarien besser geeignet, ersetzen relationale Datenbanken aber nicht vollständig.

Unterschiedliche Datenmodelle für unterschiedliche Probleme

- Key-Value für einfache schnelle Zugriffe
 - Dokumente für flexible strukturierte Daten
 - Spaltenmodelle für große analytische Datenmengen
 - Graphen für Beziehungen
 - Vektordatenbanken für KI und Ähnlichkeitssuche
-

27. Merksätze

- **Verteilte Datenhaltung basiert fast immer auf Partitionierung und Replikation.**
- **Sharding verteilt Daten, Replikation erhöht Verfügbarkeit und Fehlertoleranz.**
- **Starke Konsistenz kostet meist Latenz oder Verfügbarkeit.**
- **NoSQL ist ein Sammelbegriff für verschiedene verteilte Datenmodelle.**
- **Aggregate sind in NoSQL oft die atomare Einheit für Speicherung und Zugriff.**
- **HDFS und Ceph lösen verteilte Speicherung auf Dateisystem- bzw. Objektspeicherebene.**
- **Vektordatenbanken sind für KI-Suche und Retrieval heute zentral.**